

CricMetrics Pro: Complete File-by-File Project Architecture Guide

Easy-to-Understand Blueprint for Every File in This Project

About this Document: This guide explains every single file and folder in the **CricMetrics Pro** project repository. For every file, it clearly explains in simple English:

- **WHAT** the file is.
- **HOW** it works inside.
- **WHEN** it is used in the project.
- **WHY** it is necessary.

1. Project Directory Structure

```
/Users/laxmanpatel/Desktop/ML
├── app.py # Interactive Streamlit Web Application (6 Tabs)
├── requirements.txt # List of Python packages needed to run the project
├── README.md # Project introduction and quickstart guide for GitHub
├── FINAL_REPORT.md # Academic and viva voce project report
├── .gitignore # Tells Git which heavy files to ignore (like raw data)
├── .streamlit/
│   └── config.toml # Dark theme styling and settings for the Streamlit app
├── data/
│   ├── processed/
│   │   └── cricket_players_clean.csv # Cleaned dataset of 619 cricketers with 27 calculated stats
│   └── raw/
│       ├── matches_2008_2024.csv # 1,095 IPL match summaries (results, Player of the Match)
│       └── deliveries_2008_2024.csv # 260,920 ball-by-ball records from 17 IPL seasons (69 MB)
├── models/
│   ├── scaler.joblib # StandardScaler tool to normalize numbers (mean=0, std=1)
│   ├── knn_classifier.joblib # Best talent tier classifier (95.16% Accuracy)
│   ├── random_forest_classifier.joblib # Production tier classifier & feature importance (94.35%)
│   ├── logistic_regression.joblib # Probabilistic tier classifier (87.90% Accuracy)
│   ├── decision_tree_classifier.joblib # Visual IF-THEN flowchart classifier (87.90% Accuracy)
│   ├── kmeans_model.joblib # K-Means model grouping players into 5 tactical styles
│   ├── pca_model.joblib # PCA tool compressing 29 stats into 2D coordinates (X, Y)
│   ├── feature_metadata.json # Column names, order, and tier labels
│   └── metrics_summary.json # Test scores, cross-validation numbers, and confusion matrices
├── notebooks/
│   └── cricket_player_performance_analysis.ipynb # 21-cell Jupyter research notebook with charts
├── src/
│   ├── cricket_data_pipeline.py # Script that cleans 260k balls into 619 player rows
│   ├── train_cricket_models.py # Script that trains, tests, and saves all 6 ML models
│   └── create_cricket_notebook.py # Script that automatically builds the Jupyter notebook
├── docs/
│   ├── ML_MODELS_GUIDE.md & .pdf # Simple & easy guide to all ML models (What, When, Why, How)
│   ├── MACHINE_LEARNING_GUIDE.md & .pdf # In-depth guide to all ML topics and math formulas
│   └── PROJECT_ARCHITECTURE_GUIDE.md & .pdf # File-by-file blueprint (This document)
```

2. File-by-File Explanations

2.1 Web Application Layer

`app.py` (Interactive Streamlit Dashboard)

- **WHAT:** The main web application that coaches, franchise directors, and analysts use to explore player data and test predictions (990+ lines of Python).
 - **HOW:** Built with Streamlit, Plotly, and Pandas. It loads all trained models into memory once (`@st.cache_resource`) and displays 6 interactive tabs:
 1. 🏠 *War Room & Roster Intel:* Searchable database of 619 cricketers with filters for role, tier, and matches played.
 2. 📊 *Telemetry & Factor Impact:* Hexagonal skill radar charts (0–100), correlation bar charts, and Strike Rate vs. Average scatter plots.
 3. ⚡ *AI Rating & Auction Valuation:* Interactive sliders with 1-click presets (Virat Kohli, Jasprit Bumrah, Andre Russell, Sunil Narine, Hardik Pandya). It calculates fair player rating and estimated auction purse (₹ Crores).
 4. 🗺️ *Tactical Archetypes & 2D Map:* 2D interactive PCA scatter map of all 619 players, colored by K-Means tactical style or tier.
 5. 🎯 *What-If Franchise Simulator:* A training simulator showing how improving specific skills (like death overs strike rate) increases a player's rating and market value.
 6. 🏆 *Model Benchmarks & Defense:* Complete leaderboard comparing all models, interactive Confusion Matrix heatmaps, and Top 10 most important stats.
 - **WHEN:** Runs whenever you start the app (`streamlit run app.py`) or open the live deployment link in your browser.
 - **WHY:** Bridges complex machine learning algorithms and real-world team decision-makers. Franchise directors need clear visual tools, not raw terminal commands.
-

`.streamlit/config.toml` (Theme Settings)

- **WHAT:** Configuration file that styles the Streamlit web application.
 - **HOW:** Sets up a high-contrast dark theme with Slate Navy background (`#0F172A`) and Franchise Gold accents (`#F59E0B`).
 - **WHEN:** Loaded automatically by Streamlit before `app.py` starts.
 - **WHY:** Ensures the app looks sleek, modern, and professional on all devices and browsers, avoiding generic browser defaults.
-

2.2 Data Pipeline Layer (`src/` and `data/`)

`src/cricket_data_pipeline.py` (Data Cleaning & Feature Engineering Script)

- **WHAT:** A Python script that reads 260,920 raw ball deliveries and aggregates them into clean player-level profiles.
- **HOW:**
 1. Reads `deliveries_2008_2024.csv` and `matches_2008_2024.csv` .
 2. Calculates batting stats: runs, balls faced, strike rate, boundaries, fifties.
 3. Filters balls in overs 16–20 to compute `death_overs_strike_rate` .
 4. Calculates bowling stats: overs bowled, wickets, economy rate, and `death_overs_economy` .
 5. Counts Player of the Match awards for clutch resilience.
 6. Filters players with at least 3 matches or 20 balls faced/bowled, leaving **619 qualified cricketers**.

7. Calculates 3 composite indices: Batting Impact, Bowling Impact, and Clutch Match-Winner Index.

8. Saves the final clean table to `data/processed/cricket_players_clean.csv`.

- **WHEN:** Run whenever raw match records are updated (`python src/cricket_data_pipeline.py`).
 - **WHY:** Raw ball logs cannot be fed directly into machine learning models; they must first be converted into clean player summaries.
-

`data/processed/cricket_players_clean.csv` (Clean Analytical Dataset)

- **WHAT:** The final cleaned spreadsheet containing 619 qualified IPL cricketers and 32 columns (150 KB).
 - **HOW:** Contains 25 numerical metrics, player roles, continuous rating scores, discrete talent tiers, and estimated auction prices.
 - **WHEN:** Read by `app.py` and `train_cricket_models.py`.
 - **WHY:** Keeping this file small (150 KB) allows the cloud web app to start up in seconds, without needing to process the heavy 69 MB ball-by-ball file every time someone opens the website.
-

`data/raw/matches_2008_2024.csv` (IPL Match Records)

- **WHAT:** Summary of all 1,095 IPL matches played between 2008 and 2024 (218 KB).
 - **HOW:** Records match dates, teams, venues, match winners, and Player of the Match awards.
 - **WHEN:** Used by `cricket_data_pipeline.py` to extract clutch match-winner awards.
 - **WHY:** Player of the Match awards are official expert assessments of high-pressure impact.
-

`data/raw/deliveries_2008_2024.csv` (Raw Ball-by-Ball Data)

- **WHAT:** Every single delivery bowled in IPL history (260,920 rows, 69 MB).
 - **HOW:** Records batter, bowler, runs scored, extras, and wickets on every ball.
 - **WHEN:** Processed once by `src/cricket_data_pipeline.py`. Ignored by Git (`.gitignore`) to keep the repository lightweight.
 - **WHY:** Necessary to calculate situational metrics (like death overs strike rate in overs 16–20 and dot ball percentages) that standard scorecards do not show.
-

2.3 Machine Learning Layer (`models/` and `src/`)

`src/train_cricket_models.py` (Master Training & Evaluation Script)

- **WHAT:** The master Python script that trains, cross-validates, tests, and saves all 6 Machine Learning models (4 Classification, 1 Clustering, 1 PCA) and the StandardScaler.
- **HOW:**
 1. Loads `cricket_players_clean.csv`.
 2. Applies One-Hot Encoding on `primary_role`.
 3. Splits data into 80% training (495 players) and 20% testing (124 players).
 4. Fits `StandardScaler` on training data and scales features.
 5. Trains and cross-validates 4 classification models (KNN, Random Forest, Logistic, Decision Tree) using 5-Fold Stratified CV.
 6. Performs K-Means clustering ($k=5$) with Elbow and Silhouette scoring.
 7. Fits PCA (2 components) for 2D visualization.

8. Saves all 7 trained binary model and transformer files (`.joblib`) into `models/` and exports scores to `metrics_summary.json` .

- **WHEN:** Run during model development or retraining (`python src/train_cricket_models.py`).
- **WHY:** Centralizes all model training and testing into one automated, reproducible script.

Serialized Model Files in `models/`

File Name	Model Type	What It Does	Score in Project
<code>models/scaler.joblib</code>	StandardScaler	Rescales all 29 features to $\mu=0, \sigma=1$.	Prevents large numbers from overpowering small numbers.
<code>models/knn_classifier.joblib</code>	K-Nearest Neighbors ($k=7$)	Top Accuracy Classifier: Assigns talent tier by finding 7 similar peers.	95.16% Accuracy (Top Classifier)
<code>models/random_forest_classifier.joblib</code>	Random Forest Classifier	Ensemble of 150 trees voting on tier & feature importance.	94.35% Accuracy (94.75% 5-fold CV)
<code>models/logistic_regression.joblib</code>	Logistic Regression (Softmax)	Outputs exact risk probabilities for each tier.	87.90% Accuracy
<code>models/decision_tree_classifier.joblib</code>	Decision Tree (CART)	Visual IF-THEN flowchart for coaches.	87.90% Accuracy
<code>models/kmeans_model.joblib</code>	K-Means Clustering ($k=5$)	Groups players into 5 tactical playing styles.	Discovered 5 real tactical archetypes ($s=0.285$)
<code>models/pca_model.joblib</code>	Principal Component Analysis	Compresses 29 stats into 2 coordinates (X, Y) for 2D scatter plots.	Captures 60.79% of all information
<code>models/feature_metadata.json</code>	JSON Metadata	Stores column order, feature names, and tier labels.	Guarantees consistency between training and the web app.
<code>models/metrics_summary.json</code>	JSON Results	Stores test scores, CV scores, and confusion matrices.	Feeds the live model leaderboard in Tab 6 of <code>app.py</code> .

2.4 Research, Development & Documentation Layer

`notebooks/cricket_player_performance_analysis.ipynb` (Jupyter Research Notebook)

- **WHAT:** An interactive 21-cell data science notebook documenting the complete research study.
- **HOW:** Contains executable code cells paired with Markdown explanations:
 - *Cells 1–4:* Problem formulation, data loading, missing value handling.
 - *Cells 5–8:* Exploratory Data Analysis, correlation heatmaps, role distributions.
 - *Cells 9–12:* One-hot encoding, stratified train/test split, standard scaling.
 - *Cells 13–15:* 4-model classification benchmark (KNN, RF, Logistic, Decision Tree) with 5-Fold Stratified CV.
 - *Cells 16–17:* Confusion matrix diagnostics (KNN vs Random Forest).
 - *Cell 18:* Random Forest feature importances (Gini reduction weights).

- *Cell 19*: K-Means clustering ($k=5$) with Elbow and Silhouette charts.
 - *Cell 20*: 2D PCA scree plot and explained variance.
 - *Cell 21*: Technical Defense & Viva Voce Q&A.
 - **WHEN**: Opened in Jupyter Notebook or VS Code for interactive experimentation and grading review.
 - **WHY**: Gives professors, examiners, and data scientists a transparent, cell-by-cell walkthrough of our research.
-

`src/create_cricket_notebook.py` (Automated Notebook Generator)

- **WHAT**: A helper script that automatically builds `cricket_player_performance_analysis.ipynb` as pure JSON.
 - **HOW**: Assembles markdown text, code cells, and outputs into standard `nbformat` v4 structure.
 - **WHEN**: Run whenever the research notebook needs to be updated or refreshed (`python src/create_cricket_notebook.py`).
 - **WHY**: Ensures the notebook can be regenerated reliably without manual copy-pasting across computers.
-

`requirements.txt` (Python Package List)

- **WHAT**: A simple text file listing the exact Python libraries required to run the project:

```
`text
streamlit>=1.35.0
scikit-learn>=1.4.0
pandas>=2.1.0
numpy>=1.26.0
matplotlib>=3.8.0
seaborn>=0.13.0
plotly>=5.18.0
joblib>=1.3.0
scipy>=1.11.0
`
```

- **WHEN**: Used when setting up the environment (`pip install -r requirements.txt`) and read automatically by Streamlit Cloud during online deployment.
 - **WHY**: Guarantees that the code runs smoothly across Windows, Mac, Linux, and cloud servers.
-

`.gitignore` (Git Exclusion List)

- **WHAT**: Tells Git which files to ignore so they are not uploaded to GitHub.
 - **HOW**: Ignores the `.venv/` virtual environment, Python cache files (`__pycache__/`), and the large 69 MB raw delivery log.
 - **WHY**: Keeps the GitHub repository lightweight and prevents hitting GitHub's 100 MB file size limit, while keeping all clean datasets and trained models safely tracked.
-

`README.md` & `FINAL_REPORT.md` (Executive Project Reports)

- **WHAT**: The public GitHub homepage (`README.md`) and the comprehensive formal project defense report (`FINAL_REPORT.md`).
- **HOW**: Explains the project's background, quickstart commands, methodology proofs, and viva voce model answers.

- **WHEN:** Read by GitHub visitors, technical auditors, and academic evaluators.
- **WHY:** Clearly explains the business value of the project and satisfies the requirements of Case Study no. 102.